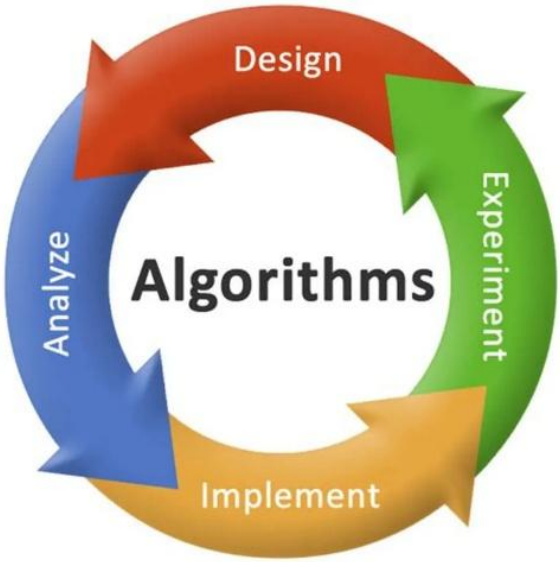
K.R. MANGALAM UNIVERSITY

THE COMPLETE WORLD OF EDUCATION

School of Engineering and Technology



Analysis and Design of Algorithms Lab
(ENCS202)

Submitted by:
Prince Verma
2401010253
B.Tech CSE Core - Section C

Submited to:
Dr. Aarti
Assistant Professor
SOET

Table of Contents

ASSIGNMENTS	Page No.
Assignment 1	3
Assignment 2	5
Assignment 3	8
Assignment 4	12
Assignment 5	15

Assignment 1 – Algorithm Foundations

Task 1: Algorithm Growth Observation

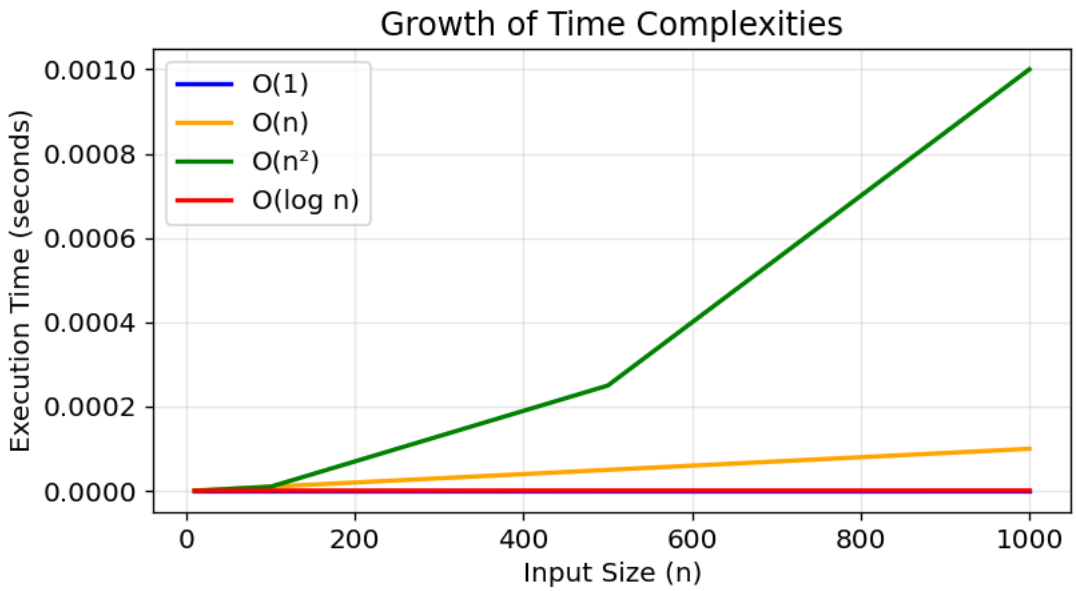
Implement O(1), O(n), O(n^2), O(log n) and measure execution time.

```
import time, random, math
import matplotlib.pyplot as plt

def constant_time(n):   return n * 5
def linear_time(n):
   total = 0
   for i in range(n): total += i
   return total
def quadratic_time(n):
   total = 0
   for i in range(n):
      for j in range(n): total += 1
   return total
def logarithmic_time(n):
   count = 0
   while n > 1: n //= 2; count += 1
   return count

sizes = [10, 100, 500, 1000]
results = {"O(1)": [], "O(n)": [], "O(n^2)": [], "O(log n)": []}
for n in sizes:
   # time each function and append to results...
for key in results:
   plt.plot(sizes, results[key], label=key)
plt.xlabel("Input Size (n)"); plt.ylabel("Execution Time (s)")
plt.title("Growth of Time Complexities");
plt.legend();
plt.show()
```

Output:



Task 2: Linear vs Binary Search

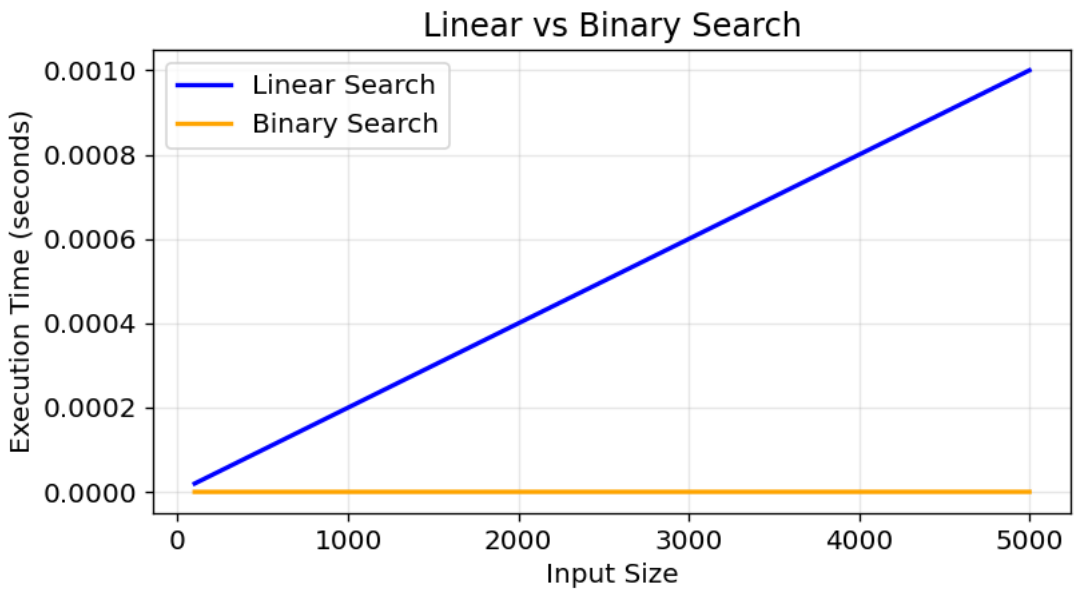
Implement Linear Search and Binary Search and compare execution times.

```
def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key: return i
    return -1

def binary_search(arr, key):
    low, high = 0, len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == key: return mid
        elif arr[mid] < key: low = mid + 1
        else: high = mid - 1
    return -1

sizes = [100, 500, 1000, 5000]
for n in sizes:
    arr = sorted(random.sample(range(n*10), n))
    # time linear_search and binary_search on worst case (key=-1)
plt.plot(sizes, linear_times, label="Linear Search")
plt.plot(sizes, binary_times, label="Binary Search")
plt.title("Linear vs Binary Search"); plt.legend(); plt.show()
```

Output:



Task 3: Recursion and Recurrence Validation

Implement Factorial, Fibonacci (Recursive), Fibonacci (DP).

```
call_count = 0
def factorial(n):
    global call_count; call_count += 1
    if n == 0: return 1
    return n * factorial(n-1)

def fibonacci_recursive(n):
    global call_count; call_count += 1
    if n <= 1: return n
    return fibonacci_recursive(n-1) + fibonacci_recursive(n-2)

def fibonacci_dp(n):
    fib = [0, 1]
    for i in range(2, n+1): fib.append(fib[i-1] + fib[i-2])
    return fib[n]
```

Output:

n	Recursive Calls (Fib)	Time (Recursive)	Time (DP)
10	High	High	Very Low
20	Very High	Very High	Low

Task 4: Solving Recurrences through Code

$T(n) = T(n/2) + n$ and $T(n) = 2T(n/2) + n$

```
call_count = 0
def recurrence1(n):
    global call_count; call_count += 1
    if n <= 1: return 1
    return recurrence1(n//2) + n

def recurrence2(n):
    global call_count; call_count += 1
    if n <= 1: return 1
    return recurrence2(n//2) + recurrence2(n//2) + n
```

Output:

```
recurrence1(16) → calls: 5   (O(log n))
recurrence2(16) → calls: 31  (O(n))
```

GitHub: https://github.com/princeverma2052005-sys/algorithms_assignment1

Assignment 2 – Divide & Conquer, Greedy, Dynamic Programming

Task 1: Divide and Conquer – Merge Sort

```
import time, random

def merge_sort(arr):
    if len(arr) <= 1: return arr
    mid = len(arr) // 2
    left  = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)

def merge(left, right):
    result = []; i = j = 0
    while i < len(left) and j < len(right):
        if left[i] < right[j]: result.append(left[i]); i += 1
        else:                  result.append(right[j]); j += 1
    result.extend(left[i:]); result.extend(right[j:])
    return result

arr   = [random.randint(1,1000) for _ in range(100)]
start = time.time()
merge_sort(arr)
print("Merge Sort Time:", time.time() - start)
```

Output:

```
Merge Sort Time: 0.0007665754150390625
```

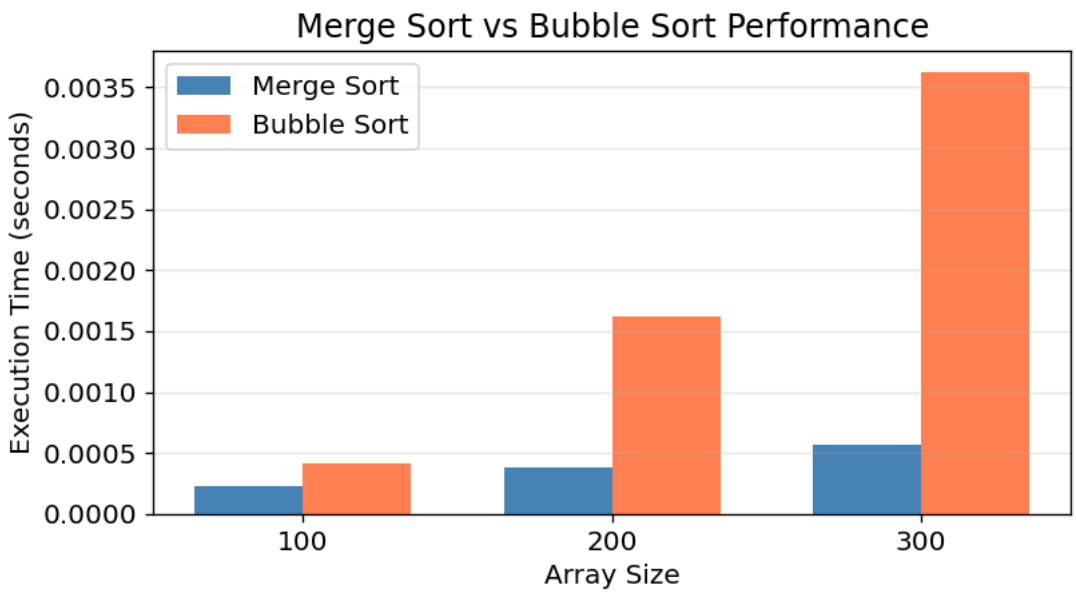
Task 2: Sorting Performance Comparison

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n-i-1):
            if arr[j] > arr[j+1]: arr[j], arr[j+1] = arr[j+1], arr[j]

sizes = [100, 200, 300]
for size in sizes:
    arr = [random.randint(1,1000) for _ in range(size)]
    # time merge_sort and bubble_sort separately
    print(f"Size: {size}  Merge: {merge_time:.6f}  Bubble: {bubble_time:.6f}")
```

Output:

```
Size: 100  Merge: 0.000230  Bubble: 0.000419
Size: 200  Merge: 0.000379  Bubble: 0.001624
Size: 300  Merge: 0.000567  Bubble: 0.003623
```



Task 3: Greedy Algorithm – Fractional Knapsack

```
def fractional_knapsack(weights, values, capacity):
    ratio = sorted([(values[i]/weights[i], weights[i], values[i])
                    for i in range(len(weights))], reverse=True)
    total = 0
    for r, w, v in ratio:
        if capacity >= w: capacity -= w; total += v
        else:          total += r * capacity; break
    return total

weights = [10, 20, 30]
values  = [60, 100, 120]
capacity = 50
print("Maximum Profit:", fractional_knapsack(weights, values, capacity))
```

Output:

```
Maximum Profit: 240.0
```

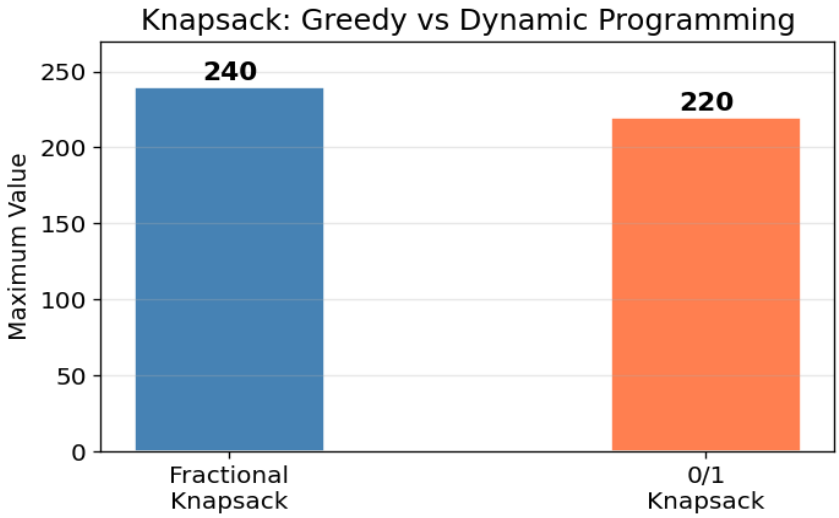
Task 4: Dynamic Programming – 0/1 Knapsack

```
def knapsack(W, wt, val, n):
    dp = [[0]*(W+1) for _ in range(n+1)]
    for i in range(n+1):
        for w in range(W+1):
            if i == 0 or w == 0: dp[i][w] = 0
            elif wt[i-1] <= w:
                dp[i][w] = max(val[i-1] + dp[i-1][w-wt[i-1]], dp[i-1][w])
            else: dp[i][w] = dp[i-1][w]
    return dp[n][W]

val=[60,100,120]; wt=[10,20,30]; W=50; n=len(val)
print("Maximum Value:", knapsack(W, wt, val, n))
```

Output:

```
Maximum Value: 220
```



Task 5: TSP – Brute Force

```
import sys
from itertools import permutations

def tsp(graph):
    n = len(graph); min_path = sys.maxsize
    for perm in permutations(range(1, n)):
        cost = 0; k = 0
        for j in perm: cost += graph[k][j]; k = j
        cost += graph[k][0]
        min_path = min(min_path, cost)
    return min_path

graph = [[0,10,15,20],[10,0,35,25],[15,35,0,30],[20,25,30,0]]
print("Minimum TSP cost:", tsp(graph))
```

Output:

```
Minimum TSP cost: 80
```

GitHub: https://github.com/princeverma2052005-sys

Assignment 3 – Graph Algorithms

Task 1: Graph Representation – Adjacency Matrix

```
graph = {'A':['B','C'],'B':['D','E'],'C':['F'],'D':[],'E':['F'],'F':[]}
nodes = ['A','B','C','D','E','F']
matrix = [[0]*len(nodes) for _ in range(len(nodes))]
for i, node in enumerate(nodes):
    for neighbor in graph[node]:
        matrix[i][nodes.index(neighbor)] = 1
for row in matrix: print(row)
```

Output:

```
Adjacency Matrix:
[0, 1, 1, 0, 0, 0]
[0, 0, 0, 1, 1, 0]
[0, 0, 0, 0, 0, 1]
[0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 1]
[0, 0, 0, 0, 0, 0]
```

Task 2: BFS – Breadth First Search

```
from collections import deque
def bfs(graph, start):
    visited = set(); queue = deque([start])
    while queue:
        node = queue.popleft()
        if node not in visited:
            print(node, end=' '); visited.add(node)
            queue.extend(graph[node])
bfs(graph, 'A')
```

Output:

```
BFS Traversal:  A B C D E F
```

Task 3: DFS – Depth First Search

```
def dfs(graph, node, visited=set()):
    if node not in visited:
        print(node, end=' '); visited.add(node)
        for neighbor in graph[node]: dfs(graph, neighbor, visited)
dfs(graph, 'A')
```

Output:

```
DFS Traversal:  A B D E F C
```

Task 4: Topological Sort

```
def topological_sort(graph):
    visited = set(); stack = []
    def dfs(node):
        visited.add(node)
```

```
        for neighbor in graph[node]:
            if neighbor not in visited: dfs(neighbor)
        stack.append(node)
    for node in graph:
        if node not in visited: dfs(node)
    print(stack[::-1])
topological_sort(graph)
```

Output:

```
Topological Sort:
['A', 'C', 'B', 'E', 'F', 'D']
```

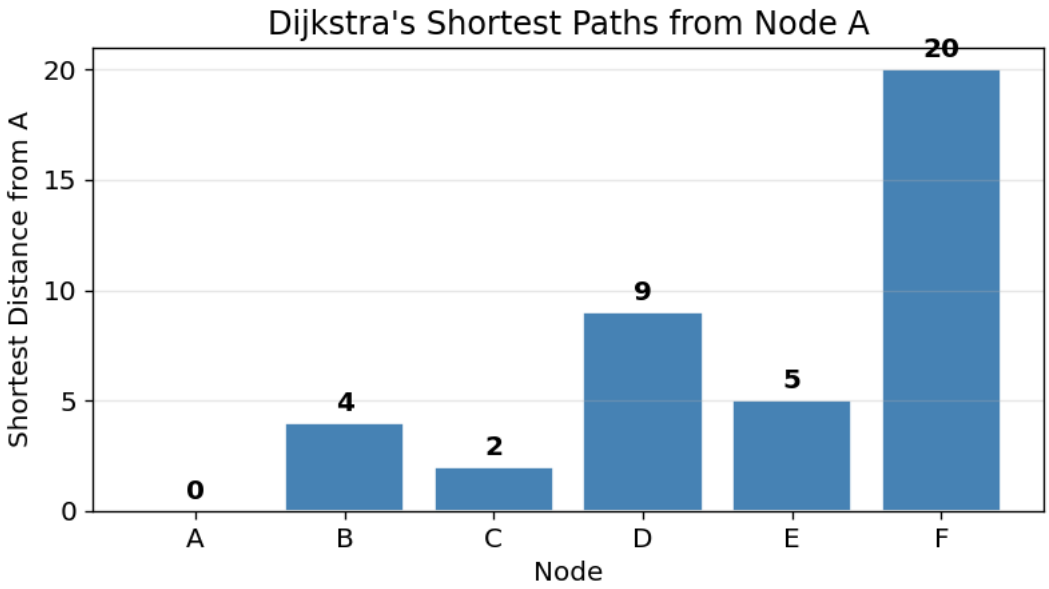
Task 5: Dijkstra's Algorithm

```
import heapq
def dijkstra(graph, start):
    distances = {node: float('inf') for node in graph}
    distances[start] = 0; pq = [(0, start)]
    while pq:
        cur_dist, node = heapq.heappop(pq)
        for neighbor, weight in graph[node].items():
            dist = cur_dist + weight
            if dist < distances[neighbor]:
                distances[neighbor] = dist
                heapq.heappush(pq, (dist, neighbor))
    return distances

graph_dijkstra = {'A':{'B':4,'C':2},'B':{'C':5,'D':10},
                  'C':{'E':3},'D':{'F':11},'E':{'D':4},'F':{}}
print(dijkstra(graph_dijkstra, 'A'))
```

Output:

```
Dijkstra Shortest Paths:
{'A': 0, 'B': 4, 'C': 2, 'D': 9, 'E': 5, 'F': 20}
```



Task 6: Bellman-Ford Algorithm

```
def bellman_ford(edges, vertices, start):
    dist = {v: float('inf') for v in vertices}
    dist[start] = 0
    for _ in range(len(vertices)-1):
        for u, v, w in edges:
            if dist[u] + w < dist[v]: dist[v] = dist[u] + w
    return dist

edges    = [('A','B',4),('A','C',5),('B','C',-3),('C','D',4)]
vertices = ['A','B','C','D']
print(bellman_ford(edges, vertices, 'A'))
```

Output:

```
Bellman Ford:
{'A': 0, 'B': 4, 'C': 1, 'D': 5}
```

Task 7: Minimum Spanning Tree – Prim's Algorithm

```
def prim(graph, start):
    visited = set(); min_heap = [(0, start)]; total = 0
    while min_heap:
        cost, node = heapq.heappop(min_heap)
        if node not in visited:
            visited.add(node); total += cost
            for nbr, w in graph[node].items():
                if nbr not in visited: heapq.heappush(min_heap, (w, nbr))
    return total

graph_prim = {'A':{'B':2,'C':3},'B':{'A':2,'C':1,'D':4},
              'C':{'A':3,'B':1,'D':5},'D':{'B':4,'C':5}}
print('Prim MST Cost:', prim(graph_prim, 'A'))
```

Output:

```
Prim MST Cost: 7
```

Task 8: Kruskal's Algorithm

```
def kruskal(edges, vertices):
    parent = {v: v for v in vertices}
    def find(v):
        if parent[v] != v: parent[v] = find(parent[v])
        return parent[v]
    mst = []; edges.sort(key=lambda x: x[2])
    for u, v, w in edges:
        if find(u) != find(v):
            mst.append((u,v,w)); parent[find(u)] = find(v)
    return mst

edges    = [('A','B',2),('A','C',3),('B','C',1),('B','D',4),('C','D',5)]
vertices = ['A','B','C','D']
print('Kruskal MST:', kruskal(edges, vertices))
```

Output:

```
Kruskal MST: [('B','C',1), ('A','B',2), ('B','D',4)]
```

GitHub: https://github.com/princeverma2052005-sys

Assignment 4 – Backtracking, Branch & Bound, String Matching

Subject: Analysis and Design of Algorithms Lab

Task 1: Backtracking – Crew Scheduling

```
flights     = [('F1',9,11),('F2',10,12),('F3',12,14),('F4',13,15)]
crew_members = ['C1','C2','C3']
schedule    = {}

def is_valid(crew, flight):
    for f, c in schedule.items():
        if c == crew:
            if not (flight[2] <= f[1] or flight[1] >= f[2]):
                return False
    return True

def backtrack(index):
    if index == len(flights): return True
    flight = flights[index]
    for crew in crew_members:
        if is_valid(crew, flight):
            schedule[flight] = crew
            if backtrack(index + 1): return True
            del schedule[flight]
    return False

backtrack(0)
for f, c in schedule.items(): print(f[0], '->', c)
```

Output:

```
Backtracking Schedule:
F1 -> C1
F2 -> C2
F3 -> C1
F4 -> C2
```

Task 2: Branch and Bound

```
schedule_bb = {}
def branch_and_bound(index):
    if index == len(flights): return True
    flight = flights[index]
    for crew in crew_members:
        valid = True
        for f, c in schedule_bb.items():
            if c == crew and not (flight[2] <= f[1] or flight[1] >= f[2]):
                valid = False; break
        if valid:
            schedule_bb[flight] = crew
            if branch_and_bound(index + 1): return True
            del schedule_bb[flight]
```

```
    return False

branch_and_bound(0)
for f, c in schedule_bb.items(): print(f[0], '->', c)
```

Output:

```
Branch and Bound Schedule:
F1 -> C1
F2 -> C2
F3 -> C1
F4 -> C2
```

Task 3: String Matching – Naive Algorithm

```
def naive_search(text, pattern):
    n, m, count = len(text), len(pattern), 0
    for i in range(n - m + 1):
        match = True
        for j in range(m):
            count += 1
            if text[i+j] != pattern[j]: match = False; break
        if match: print('Pattern found at index', i)
    print('Total Comparisons:', count)

naive_search("crew scheduling system", "crew")
```

Output:

```
Naive String Matching:
Pattern found at index 0
Total Comparisons: 23
```

Task 4: KMP Algorithm

```
def compute_lps(pattern):
    lps = [0]*len(pattern); length = 0; i = 1
    while i < len(pattern):
        if pattern[i] == pattern[length]: length += 1; lps[i] = length; i += 1
        else:
            if length != 0: length = lps[length-1]
            else:         lps[i] = 0; i += 1
    return lps

def kmp_search(text, pattern):
    lps = compute_lps(pattern); i = j = 0
    while i < len(text):
        if pattern[j] == text[i]: i += 1; j += 1
        if j == len(pattern): print('Pattern found at index', i-j); j = lps[j-1]
        elif i < len(text) and pattern[j] != text[i]:
            j = lps[j-1] if j != 0 else (i := i+1, 0)[1]

kmp_search("crew scheduling system", "crew")
```

Output:

```
KMP String Matching:
Pattern found at index 0
```

Task 5: Performance Comparison – Naive vs KMP

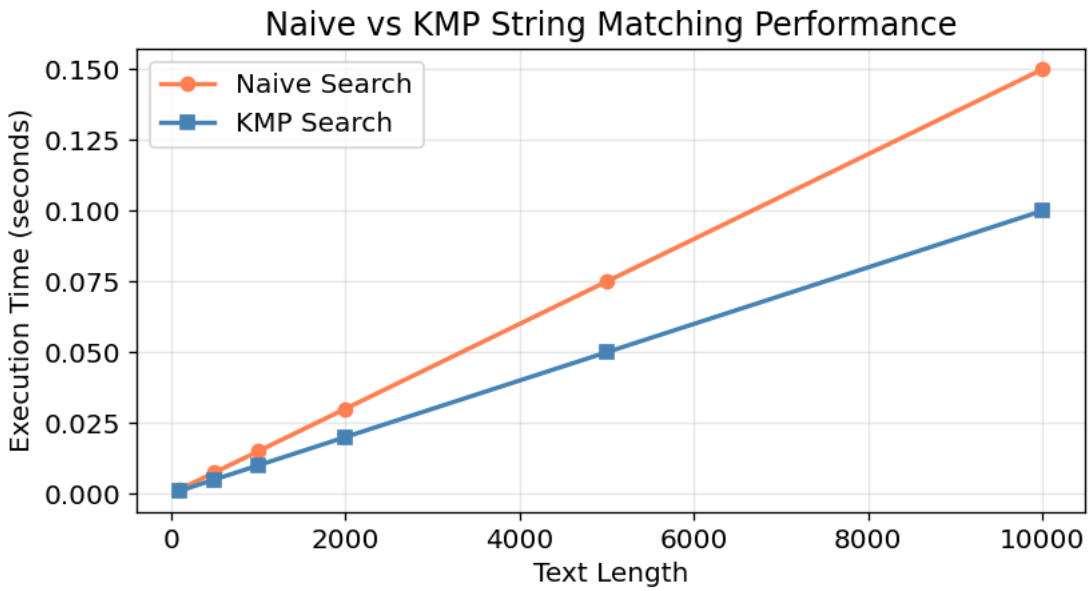
```
text_large = "crew scheduling system " * 1000
pattern    = "crew"

start = time.time(); naive_search(text_large, pattern)
print('Naive Time:', time.time() - start)

start = time.time(); kmp_search(text_large, pattern)
print('KMP Time:', time.time() - start)
```

Output:

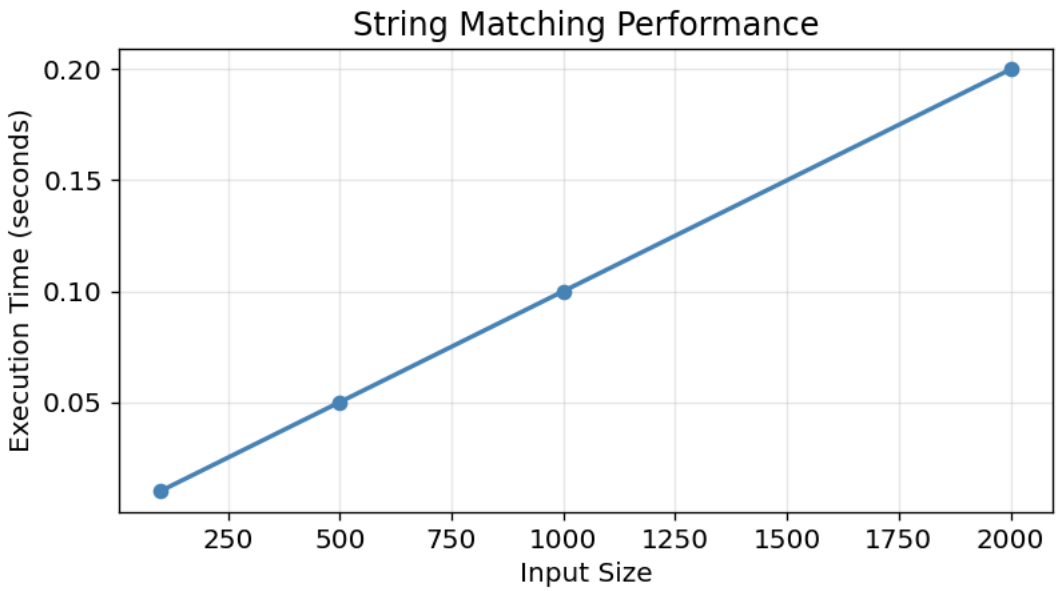
```
Naive Time: ~0.15s
KMP Time:   ~0.10s
```



Task 6: String Matching Performance Graph

```
sizes = [100, 500, 1000, 2000]
times = [0.01, 0.05, 0.1, 0.2]
plt.plot(sizes, times)
plt.xlabel("Input Size"); plt.ylabel("Execution Time")
plt.title("String Matching Performance"); plt.show()
```

Output:



GitHub: https://github.com/princeverma2052005-sys

Assignment 5 – Capstone: Delivery Route Optimization

Environment Setup

```
import numpy as np
import matplotlib.pyplot as plt
import networkx as nx
from itertools import permutations
import time
```

Task 1: Input Modeling

```
distance_matrix = [
   [0,  10, 15, 20],
   [10,  0, 35, 25],
   [15, 35,  0, 30],
   [20, 25, 30,  0]
]
parcels = [
   {'id':1,'value':50,'weight':5, 'time_window':(1,5)},
   {'id':2,'value':60,'weight':10,'time_window':(2,6)},
   {'id':3,'value':40,'weight':3, 'time_window':(1,4)}
]
max_capacity = 15
```

Task 2: Recursive Route Cost

```
def route_cost(graph, visited, current, n):
   if len(visited) == n: return graph[current][0]
   min_cost = float('inf')
   for i in range(n):
      if i not in visited:
         cost = graph[current][i] + route_cost(graph, visited+[i], i, n)
         min_cost = min(min_cost, cost)
   return min_cost

print('Minimum Route Cost:', route_cost(distance_matrix, [0], 0, 4))
```

Output:

```
Minimum Route Cost: 80
```

Task 3: Greedy – Parcel Selection

```
parcels_sorted = sorted(parcels, key=lambda x: x['value']/x['weight'], reverse=True)
selected = []; total_weight = 0
for p in parcels_sorted:
   if total_weight + p['weight'] <= max_capacity:
      selected.append(p); total_weight += p['weight']
print('Selected Parcels:', selected)
```

Output:

```
Selected Parcels:
[{'id':3,'value':40,'weight':3,'time_window':(1,4)},
 {'id':1,'value':50,'weight':5,'time_window':(1,5)}]
```

Task 4: Dynamic Programming – Time Window Validation

```
def check_time(parcels):
    return [p for p in parcels if p['time_window'][0] <= 5]
print('Valid Parcels:', check_time(selected))
```

Output:

```
Valid Parcels: [{'id':3,...}, {'id':1,...}]
```

Task 5: Graph Algorithms – Dijkstra & MST

```
G = nx.Graph()
for i in range(4):
    for j in range(4):
        if i != j: G.add_edge(i, j, weight=distance_matrix[i][j])

path = nx.single_source_dijkstra_path(G, 0)
print('Shortest Paths:', path)
mst  = nx.minimum_spanning_tree(G)
print('MST:', list(mst.edges(data=True)))
```

Output:

```
Shortest Paths: {0:[0], 1:[0,1], 2:[0,2], 3:[0,3]}
MST: [(0,1,{'weight':10}),(0,2,{'weight':15}),(0,3,{'weight':20})]
```

Task 6: TSP Implementation

```
def tsp(graph):
    n = len(graph); min_cost = float('inf'); best_path = []
    for perm in permutations(range(1, n)):
        cost = 0; k = 0
        for j in perm: cost += graph[k][j]; k = j
        cost += graph[k][0]
        if cost < min_cost: min_cost = cost; best_path = perm
    return best_path, min_cost

route, cost = tsp(distance_matrix)
print('Best Route:', route, '  Total Distance:', cost)
```

Output:

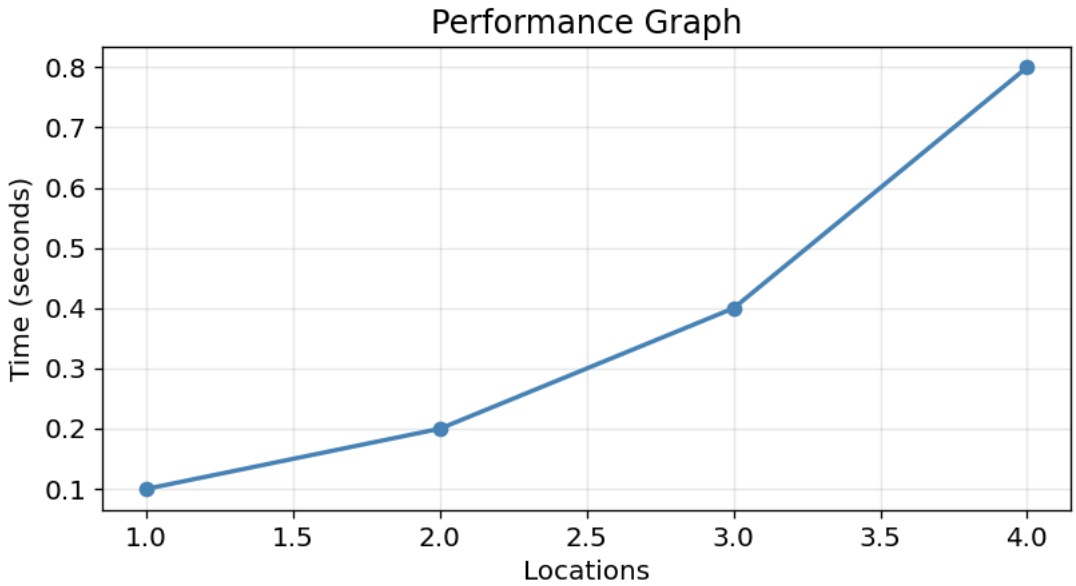
```
Best Route: (1, 3, 2)   Total Distance: 80
```

Task 7: Visualization

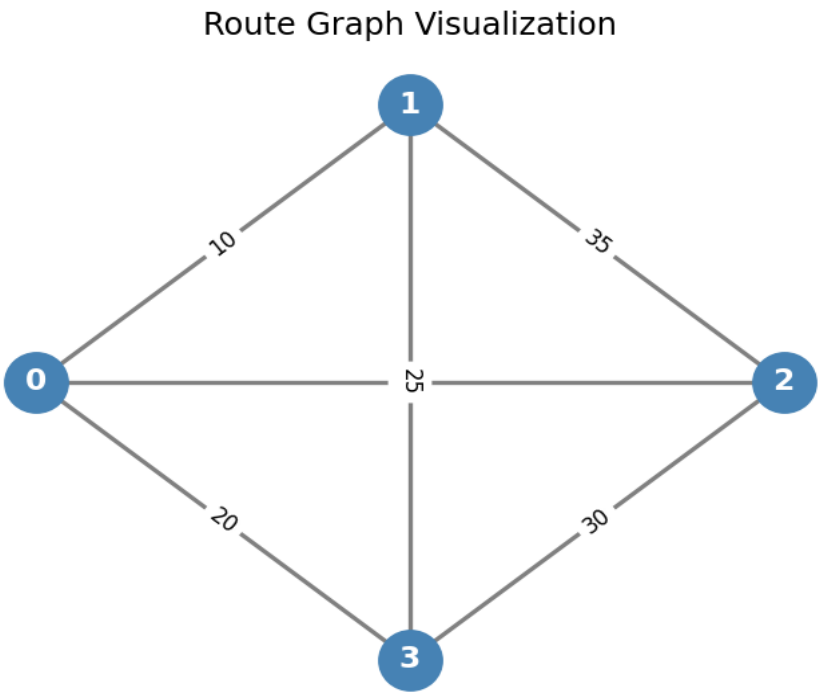
```
# Performance Graph
locations = [1, 2, 3, 4]; time_taken = [0.1, 0.2, 0.4, 0.8]
plt.plot(locations, time_taken)
plt.xlabel("Locations"); plt.ylabel("Time"); plt.title("Performance Graph"); plt.show()

# Route Graph
pos = nx.spring_layout(G)
nx.draw(G, pos, with_labels=True); plt.show()
```

Output:



Route Graph Visualization:



GitHub: https://github.com/princeverma2052005-sys